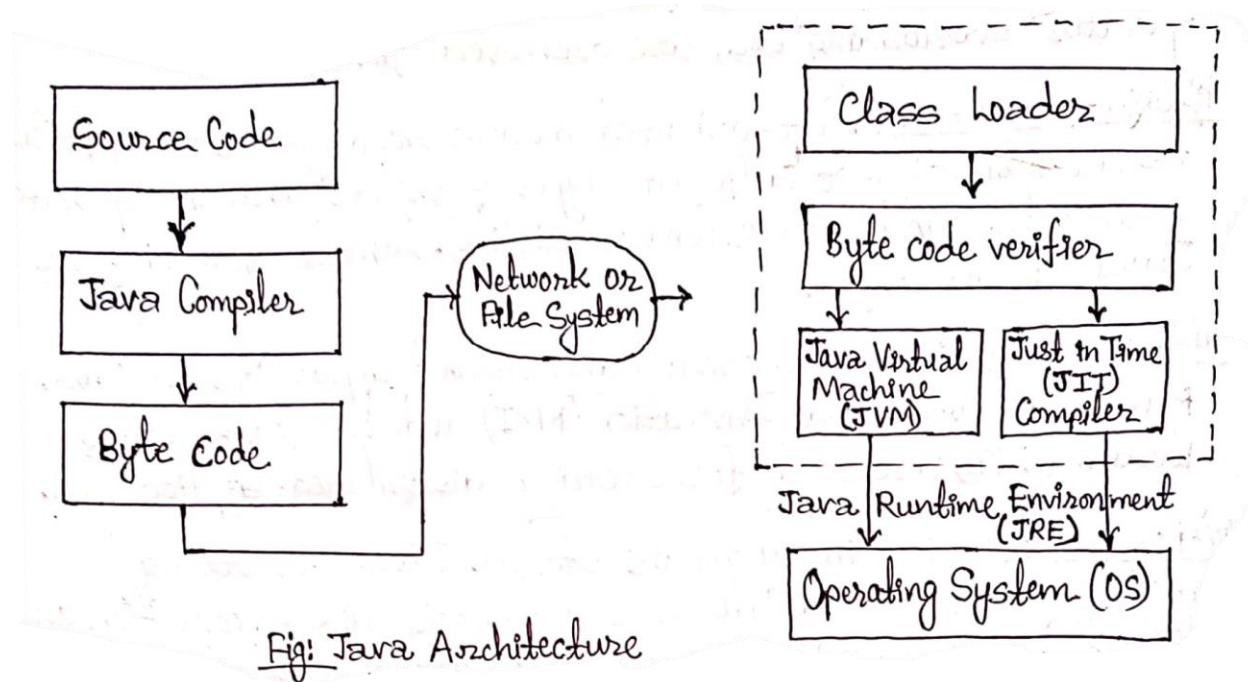


Chapter – 1

Basics of programming in java

Java Architecture



- Java source code (.java files) are compiled into byte code by java compiler.
- Byte code is not executable code for target machines; rather it is a object code for JVM. Byte code will be stored in class files (.class).
- Class loader loads all the class files required to execute the program.
- Byte code verifiers checks the byte code and ensure the following:
 - The code follows JVM specifications
 - There is no unauthorized access to memory
 - The code does not cause any stack overflows
 - There is no illegal data conversion in code such as float to object references.
- Once the code is verified, JVM will convert the byte code into machine code.
- JIT compiler helps the program execution to happen faster code is cached by JIT compiler and will be reused for the future needs.
- Finally with the help of JRE, operating systems runs the code.

Feature of java (Java Buzzword)

- a. **Object-Oriented:** Java supports the features of object-oriented programming. Its object model is simple and easy to expand.
- b. **Platform independent:** C and C++ are platform dependency languages hence the application programs written in one Operating system cannot run in any other Operating system, but in platform independence language like Java application programs written in one Operating system can able to run on any Operating system.
- c. **Simple:** Java has included many features of C / C++, which makes it easy to understand.
- d. **Secure:** Java provides a wide range of protection from viruses and malicious programs. It ensures that there will be no damage and no security will be broken.
- e. **Portable:** Java provides us the concept of portability. Running the same program with Java on different platforms is possible.
- f. **Robust:** During the development of the program, it helps us to find possible mistakes as soon as possible.
- g. **Multi-threaded:** The multithreading programming feature in Java allows you to write a program that performs several different tasks simultaneously
- h. **Distributed:** Java is designed for distributed Internet environments as it manages the TCP/IP protocol.

Path and classpath variable in Java

- Path is a mediator between developer and operating system to inform binary file path. Classpath is a mediator between developer and compiler to inform library file path those are used in our source code.
- The path points to the location of JRE. The classpath points to the classes we developed.
- We don't need to set **PATH** and **CLASSPATH** to compile and run. Java program while using IDE like Eclipse. Environment variable are required to compile and run java program using CMD.

Sample Java Program

Sample.java

```
class Sample
{
    public static void main (String a[])
    {
        System.out.println(x:"Hello world");
    }
}
```

To compile: javac Sample.java

To Execute: java Sample

How to get input from user in java

Java Scanner class allows the use to take input from the console. It belongs to java.util package

Syntax: Scanner sc = new Scanner(System.in)

Example:

```
import java.util.Scanner;

class student
{
    public static void main(String a[])
    {
        Scanner sc = new Scanner(System.in);

        System.out.println(x:"Enter your name");
        String Name = sc.nextLine();

        System.out.println(x:"Enter your roll_no");
        int Roll = sc.nextInt();

        System.out.println(Name);
        System.out.println(Roll);
    }
}
```

Data Types

Data type is a special keyword used to allocate sufficient memory space for the data. Data type is used for representing the data in main memory (RAM) of the computer. Every data type has a range of values. The compiler allocates memory space for each variable or constant according to its data type.

Two types of data types

1. Primitive data types
2. User defined data types

Primitive data types

Java defines **eight** primitive types of data: **byte, short, int, long, char, float, double, and boolean**. The primitive types are also commonly referred to as simple types. The primitive types represent single values, not complex objects. Although Java is otherwise completely object-oriented, the primitive types are not.

1. Integers

This group includes **byte, short, int, and long**, which are for whole-valued signed numbers. All of these are signed, positive and negative values. Java does not support unsigned, positive only integers. The concept of unsigned was used mostly to specify the behavior of the high order bit, which defines the sign of an integer value. The width of an integer type should not be thought of as the amount of storage it consumes, but rather as the behavior it defines for variables and expressions of that type. The width and ranges of these integer types vary widely, as shown in this table:

Name	Width	Range
long	64	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
int	32	-2,147,483,648 to 2,147,483,647
short	16	-32,768 to 32,767
byte	8	-128 to 127

```
byte num = 8;
```

2. Floating Point

This group includes **float** and **double**, which represent numbers with fractional precision. Floating-point numbers, also known as real numbers, are used when evaluating expressions that require fractional precision. For example, calculations such as square root, or transcendental such as **sine** and **cosine**, result in a value whose precision requires a floating point type. While defining float `f` should be used like `float a = 12.21f`, double does not need any kind of special notations.

Name	Width in Bits	Approximate Range
double	64	$4.9e^{-324}$ to $1.8e^{+308}$
float	32	$1.4e^{-045}$ to $3.4e^{+038}$

Double num = 5.6;

Float num = 5.6f;

3. Characters

This group includes **char**, which represents symbols in a character set, like letters and numbers. **Char** in Java is not the same as char in C or C++. In C/C++, **char** is **8 bits** wide. This is not the case in Java. Instead, Java uses Unicode to represent characters. Unicode is a unification of dozens of character sets, such as Latin, Greek, Arabic, Cyrillic, Hebrew, Katakana, Hangul, and many more. For this purpose, it requires 16 bits. Thus, in Java char is a 16-bit type. The range of a char is 0 to 65,536. There are no negative chars. The standard set of characters known as ASCII still ranges from 0 to 127 as always, and the extended 8-bit character set, ISO Latin-1, ranges from 0 to 255.

4. Boolean

This group includes **boolean**, which is a special type for representing **true/false** values. This is the type returned by all relational operators, as in the case of $a < b$. **boolean** is also the type required by the conditional expressions that govern the control statements such as if and for.

User define data types

User defined data types related variables allows us to store multiple values either of same type or different type or both. This is a data type whose variable can hold more than one value of dissimilar type, in java it is achieved using class concept.

1. Strings

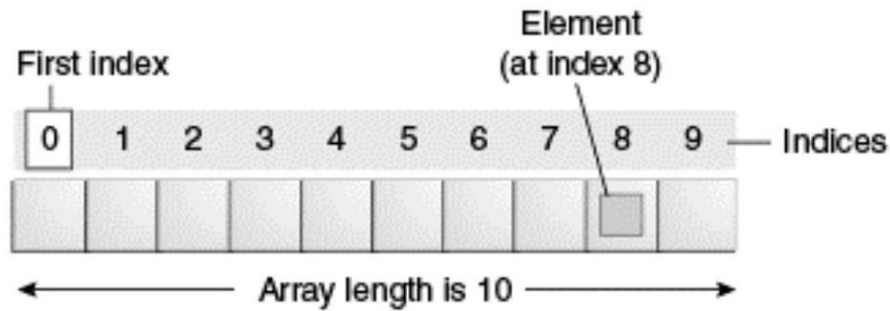
It is the combination characters or combination of characters and numbers as well. Double quote should be there while defining a string.

Syntax: = <String_Type><String_Variable> = "<sequence_of_string>";

String name = "XYZ";

2. Arrays

Java array is an object which contains elements of a similar data type. Additionally, the elements of an array are stored in a contiguous memory location. It is a data structure where we store similar elements. We can store only a fixed set of elements in a Java array. Array in Java is index-based, the first element of the array is stored at the 0th index, 2nd element is stored on 1st index and so on.



Syntax

```
dataType [] arr;
```

```
datatype arr [];
```

Instantiation of an Array in java

```
dataType [] arr = new datatype[size];
```

3. Objects

Object is an instance of a class. An object in OOPS is nothing but a self-contained component which consists of methods and properties to make a particular type of data useful. For example: color name, table, bag, barking. When you send a message to an object, you are asking the object to invoke or execute one of its methods as defined in the class.

Syntax:

```
ClassName ObjectName = new ClassName();
```

Control Statements in java

In the most basic sense, a program is a list of instructions. Control structures are programming blocks that can change the path we take through those instructions.

There are three kinds of control structures:

1. **Conditional Branches:** which we use for choosing between two or more paths. There are three types in Java: if/else/else if, ternary operator and switch.
2. **Loops/Iterations:** that are used to iterate through multiple values/objects and repeatedly run specific code blocks. The basic loop types in Java are for, while and do while.
3. **Branching Statements:** which are used to alter the flow of control in loops. There are two types in java: break and continue.

Conditional Branches

1. The if statement

The if statement is Java's conditional branch statement. It can be used to route program execution through two different paths. Here is the general form of the if statement:

```
    If (Boolean_expression)
    {
        //statements
    }
```

If the boolean expression evaluates to true, then the block of code inside the if statement will be executed. If not the first set of code after the end of the if statement (after the closing curly brace) will be executed.

Example:

```
class Test
{
    Run | Debug
    public static void main(String args[])
    {
        int x = 10;
        if (x < 20)
        {
            System.out.println(x:"This is if statement");
        }
        System.out.println(x:"Bye");
        System.out.println(x:"We are BCA students");
    }
}
```

2. The if else Statement

An if statement can be followed by an optional else statement, which executes when the Boolean expression is false.

```
    If (Boolean_expression)
    {
        // Statements 1
    }
    Else
    {
        //Statements 2
    }
```

Example

```
// WAP to find greatest number among 2 variable
class Test9
{
    Run | Debug
    public static void main(String args[])
    {
        int x = 40;
        int y = 30;
        if (x < y)
        {
            System.out.print("Greatest number is: " + y);
        }
        else
        {
            System.out.print("Greatest number is:" + x);
        }
    }
}
```

3. The if....else ifelse statement

An if statement can be followed by an optional else if...else statement, which is very useful to test various conditions using single if...else if statement. When using if , else if , else statements there are few points to keep in mind.

- An if can have zero or one else's and it must come after any else if's.
- An if can have zero to many else if's and they must come before the else.
- Once an else if succeeds, none of the remaining else if's or else's will be tested.

Syntax:

```
If (Boolean_expression1)
{
    //Executes when the Boolean expression 1 is true
}
else if (Boolean_expression 2)
{
    //Executes when the Boolean expression 2 is true
}
else if (Boolean_expression 3)
{
    //Executes when the Boolean expression 3 is true
}
else
{
    //Executes when the none of the above condition is true.
}
```

Example:

// WAP to find greatest number among three

class Test6

{

Run | Debug

public static void main(String args[])

{

int x = 60;

int y = 50;

int z = 70;

if (x>y && x>z) //false

{

System.out.print("Greatest number is: " +x);

}

else if(y>x && y>z)

{

System.out.print("Greatest number is:" +y);

}

else

{

System.out.println("Greatest number is:" +z);

}

}

}

4. Ternary operator

The ternary operator in java is a shortcut for writing if – else statements. It is also known as the conditional operator because it works based on condition. The ternary operator allows you to quickly decide between two values depending on whether a condition is true or false.

Syntax:

Condition ? trueExpression : falseExpression

Example

```
public class Ternary
{
    Run | Debug
    public static void main(String args[])
    {
        int x = 40;
        int y = 30;

        int z = 0;

        z = (x < y) ? y : x;

        System.out.println("Greatest number is " + z);
    }
}
```

5. The Switch Statement

A switch statement allows a variable to be tested for equality against a list of values. Each value is called a case, and the variable being switched on is checked for each case.

```
switch(expression)
{
    case value :
        //Statements
        break; //optional
    case value :
        //Statements
        break; //optional
//You can have any number of case statements.
    default : //Optional
        //Statements
}
```

The following rules apply to a switch statement:

- The variable used in a switch statement can only be a byte, short, int, or char.
- We can have any number of case statements within a switch. Each case is followed by the value to be compared to and a colon.
- The value for a case must be the same data type as the variable in the switch, and it must be a constant or a literal.
- When the variable being switched on is equal to a case, the statements following that case will execute until a break statement is reached.
- When a break statement is reached, the switch terminates, and the flow of control jumps to the next line following the switch statement.
- Not every case needs to contain a break. If no break appears, the flow of control will fall through to subsequent cases until a break is reached.
- A switch statement can have an optional default case, which must appear at the end of the switch. The default case can be used.

Example

```
class Test
{
    Run | Debug
    public static void main(String args[])
    {
        char grade = 'C';
        switch(grade)
        {
            case 'A' :
                System.out.println(x:"Excellent!");
                break;

            case 'B' :
                System.out.println(x:"Really Good");
                break;

            case 'C' :
                System.out.println(x:"Well done");
                break;

            case 'D' :
                System.out.println(x:"You passed");
                break;

            case 'F' :
                System.out.println(x:"Better try again");
                break;

            default :
                System.out.println(x:"Invalid grade");
        }
        System.out.println("Your grade is " + grade);
        System.out.println(x:"Bye");
    }
}
```

Loops / iterations in java

Java's iteration statements are for, while, and do-while. These statements create what we commonly call loops.

1. The While Loop

A while loop is a control structure that allows us to repeat a task a certain number of times.

Syntax:

```
while(Boolean_expression)
{
    //Statements
}
```

When executing, if the `boolean_expression` result is true then the actions inside the loop will be executed. This will continue as long as the expression result is true. Here key point of the while loop is that the loop might not ever run. When the expression is tested and the result is false, the loop body will be skipped and the first statement after the while loop will be executed.

Example

```
class Test7
{
    Run | Debug
    public static void main(String args[])
    {
        int x = 1;
        while( x < 5 )
        {
            System.out.println("value of x : " + x );
            x++;
        }
    }
}
```

2. The do while loop

A do...while loop is similar to a while loop, except that a do...while loop is guaranteed to execute at least one time.

Syntax:

```
do
{
    //Statements
}while(Boolean_expression);
```

Notice that the Boolean expression appears at the end of the loop, so the statements in the loop execute once before the Boolean is tested. If the Boolean expression is true, the flow of control jumps back up to do, and the statements in the loop execute again. This process repeats until the Boolean expression is false.

Example:

```
class Test2
{
    Run | Debug
    public static void main(String args[])
    {
        int x =10 ;
        do
        {
            System.out.println("value of x : " + x );
            x++;
        }
        while( x < 5 );
    }
}
```

3. For loop

A for loop is a repetition control structure that allows us to efficiently write a loop that needs to execute a specific number of times. A for loop is useful when we know how many times a task is to be repeated. Syntax:

```
for(initialization; Boolean_expression; update)
{
    //Statements
}
```

Example

```
class Test8
{
    Run | Debug
    public static void main(String args[])
    {
        for(int x = 0; x < 10; ++x)
        {
            System.out.println(x:"Everest Engineering College");
        }
    }
}
```

For each loop

For each loop provides an alternative approach to traverse the array or collection in java. The advantage is that it eliminates the possibility of bugs and makes code more readable. It traverse each element one by one. The drawback is that it cannot traverse elements in reverse order and cannot have option to skip any element because it does not work on an index basis. But it is recommended to use because it makes the code readable.

Syntax

```
For(data_type variableName: array / collection)
{
    //body of for – each loop
}
```

Example

```
class Test4
{
    Run | Debug
    public static void main(String args[])
    {
        String Days[] = {"sunday", "monday", "Tuesday", "wednesday", "Thursday", "Friday", "saturday"};
        for(String Day : Days )
        {
            System.out.println( Day );

            for(int i = 1; i<=4; i++)
            {
                System.out.println(i);
            }
        }
    }
}
```

Branching Statements

Branching statements are the statements used to jump the flow of execution from one part of a program to another. The branching statements are mostly used inside the control statements. Java has mainly three branching statements, i.e., continue, break, and return. The branching statements allow us to exit from a control statement when a certain condition meet.

1. Break Statements

The break keyword is used to stop the entire loop. The break keyword must be used inside any loop or a switch statement. The break keyword will stop the execution of the innermost loop and start executing the next line of code after the block.

Example:

```
class Test5
{
    Run | Debug
    public static void main(String args[])
    {
        int numbers[] = {10, 20, 30, 40, 50};
        for(int x : numbers )
        {
            if( x == 30 )
            {
                System.out.println(x:"hi");
                break;
            }
            System.out.println( x );
        }
        System.out.println(x:"Bye");
    }
}
```

2. Continue Statement

The continue keyword can be used in any of the loop control structures. It causes the loop to immediately jump to the next iteration of the loop.

- In a for loop, the continue keyword causes flow of control to immediately jump to the update statement.
- In a while loop or do/while loop, flow of control immediately jumps to the Boolean expression continue.

Example

```
public class Continue
{
    Run | Debug
    public static void main(String args[])
    {
        int numbers[] = {10, 20, 30, 40, 50};
        for(int x : numbers )
        {
            if( x == 30 )
            {
                System.out.println(x:"hi");
                continue;
            }
            System.out.println( x );
        }
        System.out.println(x:"Bye");
    }
}
```

3. Return

The return statement is also a branching statement, which allows us to explicitly return value from a method. The return statement exits us from the calling method and passes the control flow to where the calling method is invoked. Just like the break statement, the return statement also has two forms, i.e., one that passes some value with control flow and one that doesn't.

Example

```
class A
{
    int a , b , sum;
    public int add()
    {
        a = 10;
        b = 15;
        sum = a + b;
        return sum;
    }
}

class B
{
    Run | Debug
    public static void main (String args[])
    {
        A obj = new A();
        int result = obj.add();
        System.out.println("Sum of two numbers =" +result);
    }
}
```

Array in Java

Array is a collection of similar types of elements under same variable name having contiguous memory location. Arrays of any type can be created and may have one or more dimensions. A specific element in an array is accessed by its index. Arrays offer a convenient means of grouping related information.

1. One Dimensional Array

A one-dimensional array is, essentially, a list of like-typed variables. To create an array, you first must create an array variable of the desired type. The general form of a one-dimensional array declaration is

type var-name[];

Here, type declares the base type of the array. The base type determines the data type of each element that comprises the array. Thus, the base type for the array determines what type of data the array will hold. For example, the following declares an array named 'num' with the type "array of int":

int num[];

Although this declaration establishes the fact that num is an array variable, no array actually exists. In fact, the value of 'num' is set to null, which represents an array with no value. To link 'num' with an actual, physical array of integers, we must allocate one using new and assign it to num. new is a special operator that allocates memory.

int num [] = new int[4]

Hence, Obtaining an array is a two-step process.

- First, We must declare a variable of the desired array type.
- Second, we must allocate the memory that will hold the array, using new, and assign it to the array variable. Thus, in Java all arrays are dynamically allocated.

Example

```
class Array2
{
    Run | Debug
    public static void main(String a[])
    {
        int num[] = new int[4];
        num[0] = 4;
        num[1] = 8;
        num[2] = 3;
        num[3] = 9;

        for(int i = 0; i<num.length; i++)
        {
            System.out.println(num[i]);
        }
    }
}
```

2. Multi - Dimensional Array

In Java, multidimensional arrays are actually arrays of arrays. These, as we might expect, look and act like regular multidimensional arrays. To declare a multidimensional array variable, specify each additional index using another set of square brackets. For example, the following declares a two dimensional array variable called 2D.

```
int nums[][] = new int[3][4];
```

This allocates a 3 by 4 array and assigns it to nums. Internally this matrix is implemented as an array of arrays of int.

Example

```
public class Multiarray
{
    Run | Debug
    public static void main(String args[])
    {
        int nums[][] = new int[4][5];
        for(int i = 0; i<4; i++)
        {
            for(int j = 0; j<5; j++)
            {
                System.out.print(nums[i][j] +i);
            }
            System.out.println();
        }
    }
}
```

Class and Object

Java is an object – oriented programming language. Everything in java is associated with classes and objects, along with its attributes and methods. For example: A bird is class. Parrot is an object of class bird. Characteristics like color, weight, height etc are the attributes. How it flies, eat etc are its methods.

Creating class

Let us create a class called “Bird” as follows

```
public class Bird
{
    int weight = 250;
    public void myMethod()
    {
        System.out.println("Weight is: "+weight+" pounds");
    }
}
```

Creating Object

To create an object of class, we specify class name followed by object name and use the keyword new. We can create any number of object once. We create a class. Then, the attributes and methods of class can be accessed using dot(.) operators. Let us create an object for above created class Bird:

```
Bird myObj = new Bird();
```

Now we can access our class attributes and methods of Bird using myObj as follows:

```
System.out.println(myObj.weight);
```

Simple class and object program

```
class Student
{
    String name;
    int roll;
    String faculty;
}

class StudentDemo
{
    Run | Debug
    public static void main(String[] args)
    {
        Student ob = new Student();
        ob.name = "Nishanta";
        ob.roll = 10;
        ob.faculty = "BE Computer";

        System.out.println("Name = " + ob.name);
        System.out.println("Roll = " + ob.roll);
        System.out.println("Faculty = " + ob.faculty);
    }
}
```

Here in above example, we are able to access the member of student class directly from StudentDemo class. For security reasons we need to restrict the member access directly. In order to achieve this we must create private member and public function.

```
class Student1
{
    private String name;
    private int roll;
    private String faculty;

    public void setInfo() // method
    {
        name = "Nishanta";
        roll = 11;
        faculty = "BE CComputer";
    }
    public void showInfo() // method
    {
        System.out.println("Name: " + name);
        System.out.println("Roll: " + roll);
        System.out.println("Faculty: " + faculty);
    }
}
```

```
class StudentDemo1
{
    Run | Debug
    public static void main(String[] args)
    {
        Student1 ob = new Student1(); // creating instance of a class Student
        ob.setInfo(); // calling member function using object
        ob.showInfo(); // calling
    }
}
```

Constructors

It can be tedious to initialize all of the variable in a class each time an instance is created. Java allows object to initialize themselves when they are create. The automatic initialization is performed through the use of constructor.

The constructor is a special method that is used to initialize the instance variable. It is special in the sense that it does not have any return type(not even void) and its name must be exactly same as class name.

Example

```
class car
{
    private String model;
    private String Color;

    car() // constructor
    {
        model = "BMW";
        Color = "BLUE";
    }
    public void showInfo()
    {
        System.out.println("model : " + model);
        System.out.println("color :" + Color);
    }
}

class CarDemo
{
    Run | Debug
    public static void main(String args[])
    {
        car car1 = new car();
        car1.showInfo();
    }
}
```

Parameterized Constructor

The constructor which has parameter or arguments is known as parameterized constructor. In this type of constructor, we should supply / pass arguments while defining object of a class. The values of arguments are assigned to data members of the class.

Example

```
// parameterized constructor
class car
{
    private String model;
    private String Color;

    car(String m, String c) // parameterized
    constructor
    {
        model = m;
        Color = c;
    }
    public void showInfo()
    {
        System.out.println("model : " + model);
        System.out.println("color : " + Color);
    }
}

class CarDemo1
{
    Run | Debug
    public static void main(String args[])
    {
        car car1 = new car(m:"Creta",c:"red");
        car1.showInfo();
    }
}
```

Constructor Overloading

Like methods overloading, we can overload constructors for creating objects in different ways. Compiler differentiates constructors on the basis of numbers of parameters, types of the parameters and order of the parameters. Sometimes there is a need of initializing an object in different ways. This can be done using constructor overloading.

Example

```
class Student
{
    String name;
    int roll_no;
    String faculty;
    Student()
    {
        name = "Rajan";
        roll_no = 123;
        faculty = "BE Computer";
    }
    Student(String n, int r)
    {
        name = n;
        roll_no = r;
    }
    Student(String n, int r, String f)
    {
        name = n;
        roll_no = r;
        faculty = f;
    }
    public void show()
    {
        System.out.println("Name : " + name);
        System.out.println("Roll_no : " + roll_no);
        System.out.println("Faculty : " + faculty);
    }
}

class StudentDemo
{
    Run | Debug
    public static void main(String a[])
    {
        Student s1 = new Student();
        Student s2 = new Student(n:"Nishanta", r:20);
        Student s3 = new Student(n:"Pradip", r:56, f:"BE Software");

        s1.show();
        s2.show();
        s3.show();
    }
}
```

Exception Handling

- An exception is an unwanted or unexpected event, which occurs during the execution of a program i.e at run time, that disrupts the normal flow of the program's instruction.
- Exception indicates conditions that a reasonable application might try to catch.
- An exception (or exceptional event) is a problem that arises during the execution of a program. When an Exception occurs the normal flow of the program is disrupted and the program / application terminates abnormally, which is not recommended, therefore these exceptions are to be handled.
- An exception can occur for many different reasons. Following are some scenarios where an exception occurs.
 - A user has entered an invalid data.
 - A file that needs to be opened cannot be found.
 - A network connection has been lost in the middle of communication or the JVM has run out of memory.

Some of these exceptions are caused by user error, others by programmer error and others by physical resources that have failed in some manner. Based on these, we have three categories of Exceptions. You need to understand them to know how exception handling works in java.

Checked Exception

A checked exception is an exception that occurs at the compile time, these are also called as compile time exceptions. These exceptions cannot simply be ignored at the runtime of compilation the programmer should take care of (handle) these exceptions.

Example

```
class compile
{
    public static void main(String args())
    {
        System.out.Println("Nishanta teach Java and DSA in Everest");
    }
}
```

Output:

```
System.out.Println("Nishanta teach Java and DSA in Everest college");
```

^

symbol: method Println(String) |

Unchecked Exception

An unchecked exception is an exception that occurs at the time of execution. These are also called as Runtime Exceptions. These include programming bugs, such as logic errors or improper use of an API. Runtime exceptions are ignored at the time of compilation.

```
class runtime
{
    public static void main(String args[])
    {
        int i = 0;
        int j = 18/i;
        System.out.println(j);
    }
}
```

Output:

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
at runtime.main(runtime_exception.java:6)
```

Error

- Error is a subclass of the built-in class “throwable”.
- Errors are the critical condition that occurs due to the lack of the system resources and it cannot be handled by the code of the program.
- Errors cannot be recovered by any means because they cannot be created, thrown, caught or replied.
- Errors are caused due to the catastrophic failure which usually cannot be handled by your program.
- Errors are always of unchecked type, as compiler does not have any knowledge about its occurrence.
- Errors always occur in the run-time environment.
- The error can be explained with the help of an example, the program has an error of stack overflow, out of memory error, or a system crash error.

Difference between Exception and Error

Error	Exception
Error in java are of type java.lang.Error.	Exceptions in java are of type java.lang.Exception.
All errors in java are unchecked type.	Exceptions include both checked as well as unchecked type.
Errors happens at run time. They will not be known to compiler.	Checked exceptions are known to compiler where as unchecked exceptions are not known to compiler because they occur at runtime.
It is impossible to recover from errors.	We can recover from exception by handling them through try – catch blocks.
Error are mostly caused by environment in which application is running.	Exceptions are mainly caused by the application itself.
Examples: Java.lang.StackOverflowError, Java.lang.OutOfMemoryError	Examples: Checked exceptions: SQLException, IOException Unchecked exception NullPointerException, AirthmeticException

Why do we need to handle exception?

Exception handling ensures that the flow of program does not break when an exception occurs. For example, if a program has bunch of statement and an exception occurs mid way after executing certain statements then the statements after the exception will not execute and the program will terminate abruptly. By handling we make sure that all the statements execute the flow of program doesn't break.

```
class runtime
{
    public static void main(String args[])
    {
        int i = 0;
        int j = 18/i;
        System.out.println(j);

        System.out.println("Bye")
    }
}
```

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
at runtime.main(runtime_exception.java:6)
```

Here Bye statement is not print because there is an exception in statement (int j = 18 / i) ; so the statement after the exception is not printed.

How we handle the exception in java?

- To handle the exception in java we use:

Try block

- The try block contains set of statements where an exception can occur. A try block is always followed by a catch block, which handles the exception that occurs in associated try block. A try block must be followed by catch blocks and finally block or both.

Syntax

```
try block
{
    // statements that may cause an exception
}
```

Catch block

- A catch block is where you handled the exceptions; this block must follow the try block. A single try block can have several catch blocks associated with it. You can catch different exceptions in different catch blocks. When an exception occurs in try block, the corresponding catch block that handles that particular exception executes. For example if an arithmetic exception occurs in try block then the statements enclosed in catch block for arithmetic exception executes.

Syntax

Try

```
{
    // statements that may cause an exception
}
```

Catch

```
{
    // error handling code
}
```

Example

```
public class handling_exception  
{
```

Run | Debug

```
    public static void main(String args[])
```

```
    {
```

```
        int i = 0;
```

```
        int j = 0;
```

```
        try
```

```
        {
```

```
            j = 18/i;
```

```
        }
```

```
        catch(Exception e)
```

```
        {
```

```
            System.out.println("something is wrong here " +e);
```

```
        }
```

```
        System.out.println(j);
```

```
        System.out.println(x:"Bye");
```

```
    }
```

```
}
```

Multiple Catch Block

A try block can be followed by one or more catch blocks. Each catch block must contain a different exception handler. So, if you have to perform different tasks at the occurrence of different exceptions, use java multi-catch block.

Syntax

```
Try
{
    // protected code
}
Catch(ExceptionType1 e)
{
    // catch block
}
Catch(ExceptionType2 e)
{
    // catch block
}
Catch(ExceptionType3 e)
{
    // catch block
}
```

Example

```
public class multiple_catch
{
    Run | Debug
    public static void main(String args[])
    {
        int i = 2;
        int j = 0;

        int num[] = {1, 2, 3, 4, 5};
        String str = null;

        try
        {
            j = 18/i;
            System.out.println(num[7]);
            System.out.println(str.length());
        }
        catch(ArithmeticException e)
        {
            System.out.println("cannot divide by zero" +e);
        }
        catch(ArrayIndexOutOfBoundsException e)
        {
            System.out.println(x:"Array is out of stock" );
        }
        catch(Exception e)
        {
            System.out.println("something went wrong " +e);
        }
        System.out.println(j);
        System.out.println(x:"Bye");
    }
}
```

User Defined Exception

- If you are creating your own exception that is known as user defined exception.
- Java user defined exception are used to customize the exception according to user need.
- In java we can create our own exception class and throw that exception using throw keyword.
- An exception is nothing but an object of an exception class. We can create our exception in the following ways.
 - By creating a class which extends the superclass of all the exception classes i.e Exception class
 - By creating a class which extends any of the subclasses of the Exception class, such as `ArrayStoreException`, `IOException`, `ParseException`, `NullPointerException` etc.

Example

```
class NishantaException extends Exception
{
    public NishantaException(String str)
    {
        super(str);
    }
}

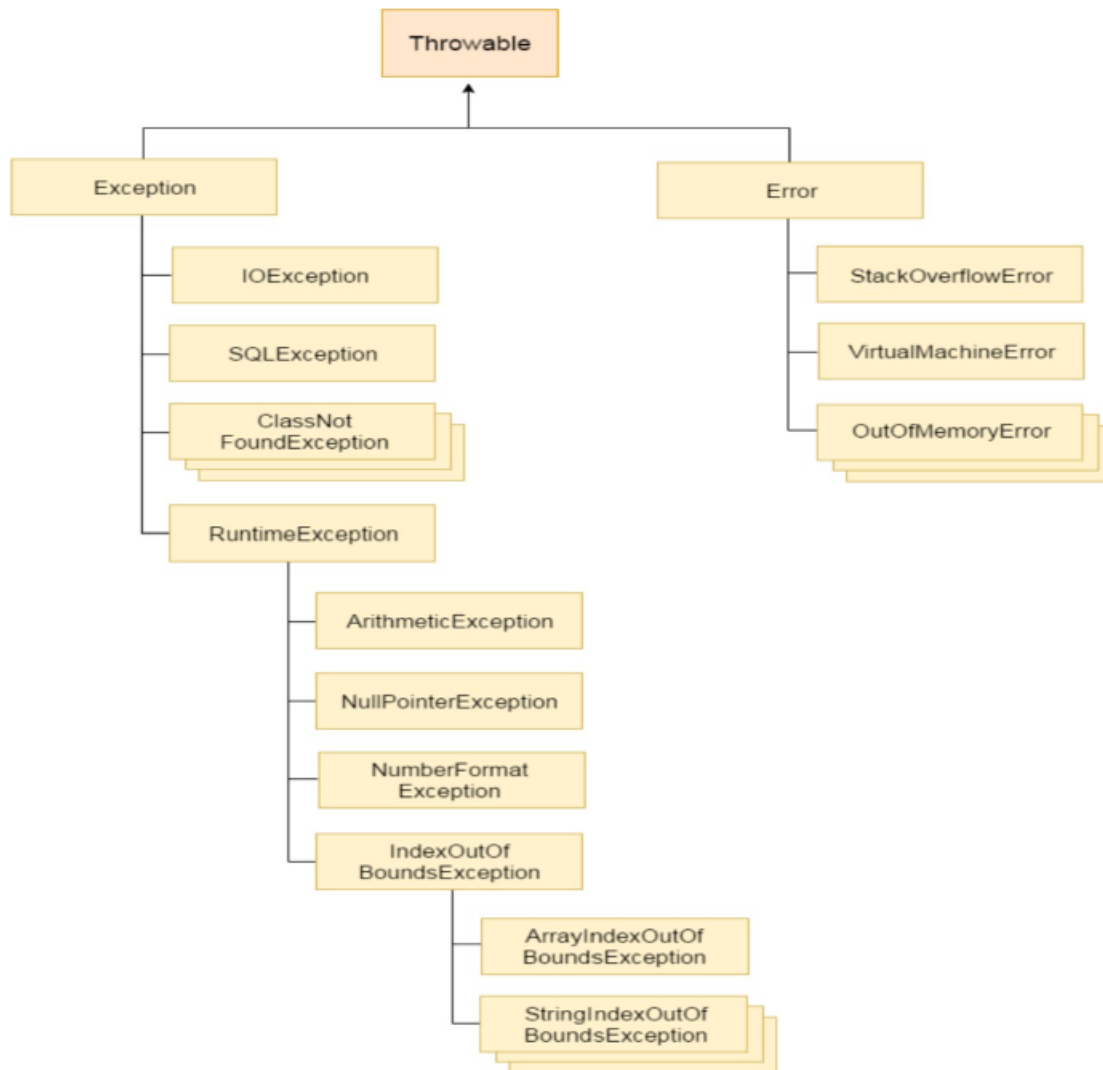
class user_exception
{
    Run | Debug
    public static void main(String args[])
    {
        int i = 20;
        int j = 0;

        try
        {
            j = 18/i;
            if(j == 0)
                throw new NishantaException(str:"I dont want to print zero");
        }
        catch(NishantaException e)
        {
            j = 18 / 1;
            System.out.println("thats the default value " +e);
        }

        System.out.println(j);
        System.out.println(x:"Bye");
    }
}
```

Hierarchy of Java Exception Classes

The `java.lang.Throwable` class is the root class of java Exception hierarchy which is inherited by two subclasses: `Exception` and `Error`. A hierarchy of java exception classes are given below:



Throw statement

- However, it is possible for our program to throw an exception explicitly, using the **throw** statement. The general form of throw is shown here:

throw ThrowableInstance;

- Here, *ThrowableInstance* must be an object of type *Throwable* or a subclass of *Throwable*. Primitive types, such as *int* or *char*, as well as *non-Throwable* classes, such as *String* and *Object*, cannot be used as exceptions.
- There are two ways that we can obtain a *Throwable* object: using a parameter in a *catch* clause, or creating one with the new operator.
- The flow of execution stops immediately after the throw statement; any subsequent statements are not executed.
- The nearest enclosing try block is inspected to see if it has a catch statement that matches the type of exception.
- If it does find a match, control is transferred to that statement. If not, then the next enclosing try statement is inspected, and so on. If no matching catch is found, then the default exception handler halts the program and prints the stack trace.

Syntax

throw Exception

Example

```
class Throw
{
    Run | Debug
    public static void main(String args[])
    {
        int i = 20;
        int j = 0;

        try
        {
            j = 18/i;
            if(j == 0)
                throw new ArithmeticException(s:"I dont want to print zero");
        }
        catch(ArithmeticException e)
        {
            j = 100;
            System.out.println("cannot divide by zero " +e);
        }

        System.out.println(j);

        System.out.println(x:"Bye");
    }
}
```

Throws keyword / Ducking Exception

- Java throws keyword is used to declare an exception.
- It gives an information to the programmer that there may occur an exception so it is better for the programmer to provide the exception handling code so that normal flow can be maintained.
- The throws is a keyword in java which is used in the signature of method to indicate that his method might throw one of the listed type exception.
- The caller to these methods has to handle the exception using a try – catch block.

Syntax:

return_type method name () throws exception _class_name

```
{  
    // method code  
}
```

Example:

```
class calculate  
{  
    public void division(int a, int b) throws ArithmeticException  
    {  
        int d = a/b;  
        System.out.println(d);  
    }  
}
```

```
class Throws  
{  
    Run | Debug  
    public static void main(String args[])  
    {  
        calculate calc = new calculate();  
        try  
        {  
            calc.division(a:10,b:0);  
        }  
        catch (ArithmeticException e)  
        {  
            System.out.println(x:"Divide by zero is not possible");  
        }  
    }  
}
```

Difference between Throw and Throws

Throw	Throws
It is used to create a new Exception object and throw it.	It is used in method definition, to declare that a risky method is being called.
Using throw keyword you can declare only one Exception at a time.	Using throws keyword you can declare multiple exception at a time.
Throw keyword is used to throw an exception explicitly.	Throws keyword is used to declare one or more exceptions separately by commas.
Throw keyword is used within the method.	Throws keyword is used with the method signature.
Syntax wise throw keyword is followed by the instance variable.	Syntax wise throws keyword is followed by exception class name.
Checked exception cannot be propagated using throw only. Unchecked exception can be propagated using throw.	For the propagation checked exception must use throws keyword followed by specific exception class name.
Example: throw new IOException("cannot open connection").	Example: Throws IOException, ArrayIndexOutOfBoundsException

Finally Keyword

- A finally block contains all the crucial statements that must be executed whether exception occurs or not.
- The statements present in this block will always execute regardless of whether exception occurs in try block or not such as closing a connection, stream etc.
- The finally block always executes when the try block exists. This ensures that the finally block is executed even if an unexpected exception occurs.
- But finally is useful for more than just exception handling. It allows the programmer to avoid having cleanup code accidentally bypassed by a return, continue or break

Syntax

Try

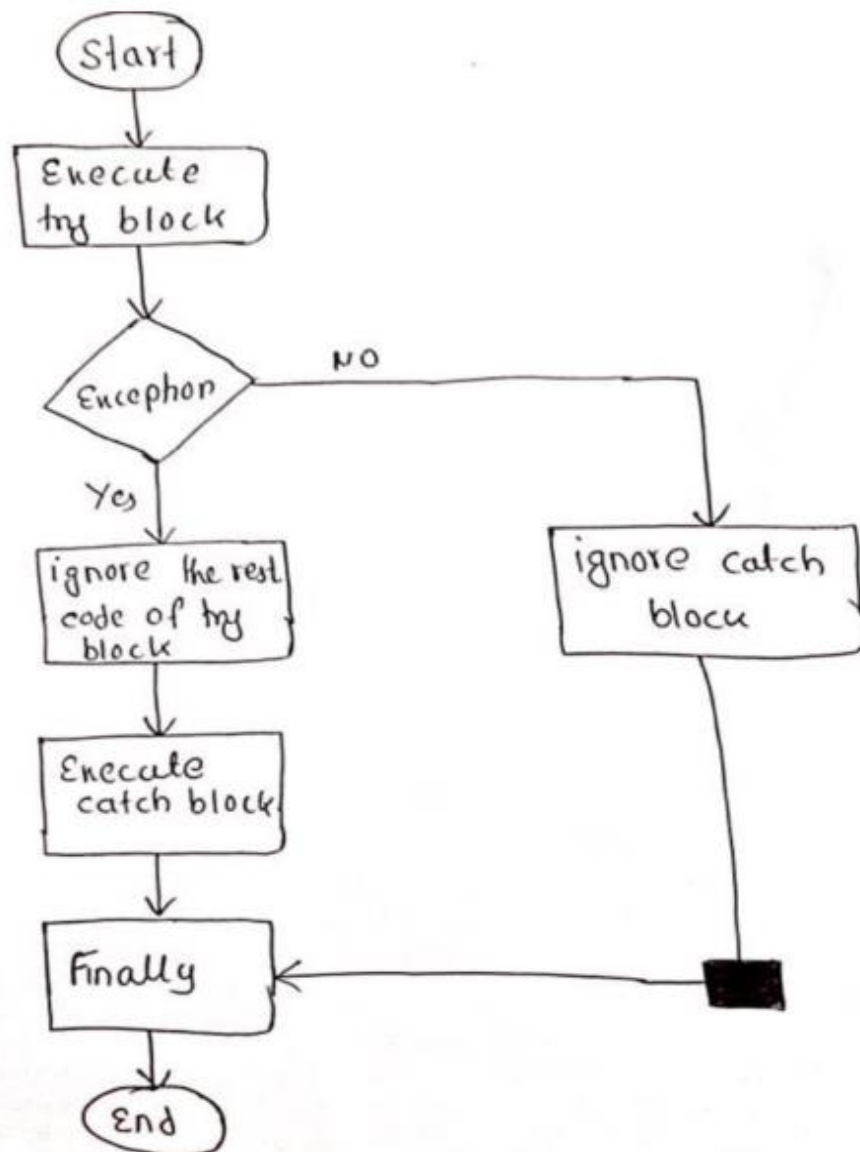
```
{  
    //Statement that may cause an exception  
}
```

Catch

```
{  
    // handling exception  
}
```

Finally

```
{  
    // statement to be executed  
}
```



Example

```
class Finally
{
    Run | Debug
    public static void main(String args[])
    {
        int i = 0;
        int j = 0;

        int num[] = {1, 2, 3, 4, 5};

        try
        {
            System.out.println(num[1]);
            j = 18/i;
            System.out.println(j);
        }
        catch(ArrayIndexOutOfBoundsException e)
        {
            System.out.println(x:"Data is out of array");
        }
        finally
        {
            System.out.println(x:"we complete exception handling chapter");
        }
        System.out.println(x:"You guys have to do assignment");
    }
}
```

Collection Classes / Interfaces

- A collection is a group of individual objects represented as a single unit. Java provides Collection Framework which defines several classes and interfaces to represent a group of objects as a single unit.
- The collection interface (**java.util.Collection**) and map interface (**java.util.Map**) are the two main “root” interfaces of java collection classes.

Advantages of Collection Framework:

- Before collection framework (or before JDK 1.2) was introduced, the standard methods for grouping java objects (or collections) were Arrays or Vectors or Hashtables. All of these collections had no common interface.
- Accessing elements of these Data Structures was a hassle as each had a different method (and syntax) for accessing its members.
 - a. Consistent API: The API has a basic set of interfaces like collection, set, List or Map. All classes (ArrayList, LinkedList, Vector etc) that implements these interfaces have some common set of methods.
 - b. Reduces programming effort: A programmer doesn't have to worry about the design of collection and he can focus on its best use in his program.
 - c. Increases program speed and quality: Increases performance by providing high performance implementations of useful data structures and algorithms.

Collection Interfaces

1. Map Interface

The **java.util.Map** interface represents a mapping between a key and a value. Few characteristics of the Map Interface are:

- A Map cannot contain duplicate keys and each key can map to at most one value. Some implementations allow null key and null value like the HashMap and LinkedHashMap but some do not like the TreeMap.
- The order of a map depends on specific implementations e.g TreeMap and LinkedHashMap have predictable order, while HashMap does not.
- There are two interfaces for implementing Map in java: Map and SortedMap and three classes: HashMap, TreeMap and LinkedHashMap.

Methods in Map Interface:

- a. **put(key, value):** This method is used to insert an entry in this map.
- b. **putAll(Map map):** This method is used to insert the specified map in this map.
- c. **remove(key):** This method is used to delete an entry for the specified key.
- d. **get(key):** This method is used to return the value for the specified key.
- e. **containskey(key):** This method is used to search the specified key from this map.
- f. **clear():** This method clear all the elements.
- g. **size():** This method returns the number of components.
- h. **Keyset():** Gives all the keys.

Example:

```
class MapInterface
```

```
{
```

Run | Debug

```
public static void main(String args[])
```

```
{
```

```
    Map<String, Integer> students = new HashMap<>();
```

```
    students.put(key:"Sudip", value:56);
```

```
    students.put(key:"Chesta", value:23);
```

```
    students.put(key:"Keshab", value:67);
```

```
    students.put(key:"nainesh", value:98);
```

```
    students.put(key:"nainesh", value:72);
```

```
    students.put(key:"Nishant", value:28);
```

```
    System.out.println(students);
```

```
    System.out.println(students.get(key:"Sudip"));
```

```
    System.out.println(students.keySet());
```

```
    System.out.println(students.values());
```

```
    for(String key : students.keySet())
```

```
    {
```

```
        System.out.println(key + ": " +students.get(key));
```

```
    }
```

```
}
```

```
}
```

Output:

```
{Keshab=67, Sudip=56, nainesh=72, Chesta=23}
```

```
56
```

```
[Keshab, Sudip, nainesh, Chesta]
```

```
[67, 56, 72, 23]
```

```
Keshab: 67
```

```
Sudip: 56
```

```
nainesh: 72
```

```
Chesta: 23
```

2. List interface:

The java.util.List is a child interface of collection. It is an ordered collection of objects in which duplicate values can be stored. Since list preserves the insertion order, it allows positional access and insertion of elements. List interface is implemented by ArrayList, LinkedList, Vector and Stack Classes.

List is an interface and the instances of List can be created in the following ways:

```
List a = new ArrayList();
```

```
List b = new LinkedList();
```

```
List c = new Vector();
```

```
List d = new Stack();
```

Methods in List Interface:

- a. add(element): This method is used to add elements in list.
- b. get(index): This method is used to retrieve elements on the basis of index number.
- c. isEmpty(): This method tests if this list has no components.
- d. remove(index): This method removes a single elements on the basis of index number.
- e. clear(): This method clear all the elements from list.
- f. size(): This method returns the number of components in this list.
- g. removeAllElements(): This method removes all components from this list and sets its size to zero.
- h. indexOf(element): This method returns first occurrence of given element or -1 if element is not present in list.
- i. lastIndexOf(element): This method returns the last occurrence of given element or -1 if element is not present in list.

Example:

```
import java.util.ArrayList;
import java.util.List;

class ListInterface
{
    Run | Debug
    public static void main (String args[])
    {
        List<Integer> li = new ArrayList<Integer>();
        //adding items
        li.add(e:10);
        li.add(e:20);
        li.add(e:30);
        li.add(e:40);
        li.add(e:5);
        li.add(e:40);

        // retrieving items
        System.out.println(li);
        System.out.println("item at index 1 is: "+li.get(index:1));

        //removing item
        li.remove(index:2);
        System.out.println("After removing item: "+ li);

        //getting size
        System.out.println("Size of list: "+ li.size());

        // removing all items
        li.clear();
        System.out.println("After removing all items: "+li);
    }
}
```

Output:

```
[10, 20, 30, 40, 5]
item at index 1 is: 20
After removing item: [10, 20, 40, 5]
Size of list: 4
After removing all items: []
```

3. Set Interface:

- Set is an interface which extends collection. It is an unordered collection of objects in which duplicate values cannot be stored.
- Basically, Set is implemented by HashSet, LinkedHashSet or TreeSet (Sorted representation).
- Set has various methods to add, remove clear, size etc to enhance the usage of this interface.

Methods in Set interface:

- a. Add(element): This method is used to add element in set.
- b. Get(index): This method is used to retrieve elements on the basis of index number.
- c. IsEmpty(): This method tests if this set has no components.
- d. Remove(index): This method removes a single element on the basis of index number.
- e. Clear(): This method clear all the elements from set.
- f. Size(): This method returns the number of components in this set.
- g. RemoveAllElements(): This method removes all components from this set and sets its size to zero.
- h. IndexOf(element): This method returns first occurrence of given element or -1 if element is not present in set.
- i. LastIndexOf(element): This method returns the last occurrence of given element or -1 if element is not present in set.

Example:

```
import java.util.Set;
import java.util.TreeSet;
```

```
class SetInterface
{
```

Run | Debug

```
public static void main(String args[])
{
    //Set<String> s1 = new HashSet<String>();
    Set<String> s1 = new TreeSet<String>();
    s1.add(e:"Keshab");
    s1.add(e:"sudip");
    s1.add(e:"preshu");
    s1.add(e:"sushan");
    s1.add(e:"sudip");

    System.out.println(s1);
    System.out.println(s1.size());
    //System.out.println(s1.remove("sudip"));
    //System.out.println(s1);
}
```

Output

Set in Hashset:

[Keshab, sudip, sushan, preshu]

Set in TreeSet:

[Keshab, preshu, sudip, sushan]

Collection classes:

1. Array List:

- The ArrayList class extends Abstract List and implements the List interface.
- ArrayList supports dynamic arrays that can grow as needed.
- Standard java arrays are of a fixed length. After arrays are created, they cannot grow or shrink, which means that you must know in advance how many elements an array will hold.
- Array lists are created with an initial size. When this size is exceeded, the collection is automatically enlarged. When objects are removed, the array may be shrunk.

Methods of ArrayList

add(), get(), remove(), size(), clear(), isEmpty(), indexOf(), Contains(), get()

Example:

```
import java.util.ArrayList;
class Arraylist1
{
    Run | Debug
    public static void main(String args[])
    {
        ArrayList<String> al = new ArrayList<String>();
        System.out.println("Initial size of array list: "+al.size());

        // add elements to the array list
        al.add(e:"N");
        al.add(e:"I");
        al.add(e:"S");
        al.add(e:"H");
        al.add(e:"A");
        al.add(e:"N");
        al.add(e:"T");
        al.add(e:"A");
        al.add(e:"A2");

        System.out.println("Size of array list after addition: "+al.size());

        // display the array list
        System.out.println("content of array list:" +al);

        // Remove elements from the array list
        al.remove(o:"A");
        al.remove(index:7);
        System.out.println("size of array list after deletion: "+al.size());
        System.out.println("content of arraylist: "+al);
    }
}
```

Output:

Initial size of array list: 0

Size of array list after addition: 9

content of array list:[N, I, S, H, A, N, T, A, A2]

size of array list after deletion: 7

content of arraylist: [N, I, S, H, N, T, A]

2. Linked List:

- Like arrays, linked list is a linear data structure. Unlike arrays, linked list elements are not stored at the contiguous location, the elements are linked using pointer.
- Methods are same as arraylist.

Example:

```
import java.util.LinkedList;

class Linkedlist1
{
    Run | Debug
    public static void main(String args[])
    {
        LinkedList<String> li = new LinkedList<String>();
        System.out.println("Initial size of list: "+li.size());

        // add elements to the array list
        li.add(e:"N");
        li.add(e:"I");
        li.add(e:"S");
        li.add(e:"H");
        li.add(e:"A");
        li.add(e:"N");
        li.add(e:"T");
        li.add(e:"A");
        li.add(index:3,element:"A2");
        li.add(e:"keshab and Nishant");

        System.out.println("Size of array list after addition: "+li.size());

        // display the array list
        System.out.println("content of array list:" +li);

        // Remove elements from the array list
        li.remove(o:"I");
        li.remove(index:2);
        li.remove(index:7);
        System.out.println("size of array list after deletion: "+li.size());
        System.out.println("content of arraylist: "+li);
    }
}
```

Output:

Initial size of list: 0

Size of array list after addition: 10

content of array list:[N, I, S, A2, H, A, N, T, A, keshab and Nishant]

size of array list after deletion: 7

content of arraylist: [N, S, H, A, N, T, A]

Hash Map

- HashMap is a part of java collection since java 1.2.
- It provides the basic implementation of Map interface of java. It stores the data in (Key, Value) pairs.
- To access a value one must know its key. HashMap is known as Hash Map because it uses a technique called hashing.
- Hashing is a technique of converting a large string to a small string that represents the same string.
- A shorter value helps in indexing and faster searches.
- Methods are the same as Hash set.

Example:

```
import java.util.HashMap;

public class HashMap1
{
    Run | Debug
    public static void main(String args[])
    {
        HashMap<Integer, String> map = new HashMap<>();
        map.put(key:10, value:"Messi");
        map.put(key:7, value:"CR 7");
        map.put(key:12, value:"marcelo");

        System.out.println(map);
        System.out.println("size of map is: "+map.size());

        if (map.containsKey(key:12))
        {
            String st = map.get(key:12);
            System.out.println("value for key 12 is: "+st);
        }

        System.out.println(map.keySet());
        System.out.println(map.values());

        map.clear();
        System.out.println("after clearing map: "+map);
    }
}
```

Output:

```
{7=CR 7, 10=Messi, 12=marcelo}
size of map is: 3
value for key 12 is: marcelo
[7, 10, 12]
[CR 7, Messi, marcelo]
after clearing map: {}
```

Hash Set

- The HashSet class implements the set interface, backed by a hash table which is actually a HashMap instance. Few important features of HashSet are
- Implements Set interface.
- Underlying data structures for HashSet is hashtable.
- As it implements the set interface, duplicate values are not allowed.
- Objects that you insert in HashSet are not guaranteed to be inserted in same order.
- Objects are inserted based on their hash key.
- NULL elements are allowed in HashSet.
- Methods are similar to map set interface.

Example:

```
import java.util.HashSet;

public class Hash_Set
{
    Run | Debug
    public static void main(String args[])
    {
        HashSet<String> hs = new HashSet<String>();

        // Adding elements into Hashset
        hs.add(e:"India");
        hs.add(e:"Nepal");
        hs.add(e:"Pakistan");
        hs.add(e:"Bangladesh");
        hs.add(e:"Afganistan");
        hs.add(e:"India");

        //Displaying the Hashset
        System.out.println(hs);
        System.out.println("List contains India or not: "+hs.contains(o:"India"));

        // Removing items from HashSet using remove()
        hs.remove(o:"Pakistan");
        System.out.println("List after removing pakistan: "+hs);
    }
}
```

Output:

[Bangladesh, Pakistan, Afganistan, Nepal, India]

List contains India or not: true

List after removing pakistan: [Bangladesh, Afganistan, Nepal, India]

Tree Set

TreeSet is one of the most important implementations of the SortedSet interface in java that uses a Tree for storage. The ordering of the elements is maintained by a set using their natural ordering whether or not an explicit comparator is provided. This must be consistent with equals if it is to correctly implement the Set interface.

Few important features of TreeSet are as follows:

- TreeSet implements the SortedSet interface so duplicate values are not allowed.
- Objects in a TreeSet are stored in a sorted and ascending order.
- TreeSet does not preserve the insertion order of elements but elements are sorted by keys.
- TreeSet does not allow to insert Heterogeneous objects. It will throw classCastException at Runtime if trying to add heterogeneous objects.
- TreeSet serves as an excellent choice for storing large amounts of sorted information which are supported to be accessed quickly because of its faster access and retrieval time.

Methods are same as set interface.

Example:

```
import java.util.TreeSet;

public class Treeset
{
    Run | Debug
    public static void main(String args[])
    {
        TreeSet<String> Ts = new TreeSet<String>();

        // Adding elements into Treeset
        Ts.add(e:"India");
        Ts.add(e:"Nepal");
        Ts.add(e:"Pakistan");
        Ts.add(e:"Bangladesh");
        Ts.add(e:"Afganistan");
        Ts.add(e:"India");

        //Displaying the Treeset
        System.out.println(Ts);
        System.out.println("List contains India or not: "+Ts.contains(o:"India"));

        // Removing items from TreeSet using remove()
        Ts.remove(o:"Pakistan");
        System.out.println("List after removing pakistan: "+Ts);
    }
}
```

Output:

[Afganistan, Bangladesh, India, Nepal, Pakistan]

List contains India or not: true

List after removing pakistan: [Afganistan, Bangladesh, India, Nepal]

This Keyword

- a. This keyword refers to the current object inside a method or Constructor

```
class A
{
    void show()
    {
        System.out.println("Output of this: " +this);
    }
    Run | Debug
    public static void main(String args[])
    {
        A ob = new A();
        System.out.println("output of object: "+ob);
        ob.show();
    }
}
```

- b. Whenever the name of instance and local variable both are same then our runtime environment JVM gets confused that which one is local variable and which one is instance variable, to avoid this problem we should use 'this' keyword

```
class A
{
    int a; // instance varriable
    A (int a) // local variable
    {
        this.a = a;
    }
    void show()
    {
        System.out.println(a);
    }
    Run | Debug
    public static void main(String args[])
    {
        A r = new A(a:10);
        r.show();
    }
}
```

c. It is also used when we want to call the default constructor of its own class.

```
class A
{
    A ()
    {
        System.out.println(x:"we are BCA students");
    }
    A(int a)
    {
        this();
        System.out.println(a);
    }
    Run | Debug
    public static void main(String args[])
    {
        A r = new A(a:100);
    }
}
```

d. It is also used to call parameterized constructor of its own class

```
class A
{
    A ()
    {
        this(a:10);
    }
    A(int a)
    {
        System.out.println(a);
    }
    Run | Debug
    public static void main(String args[])
    {
        A r = new A();
    }
}
```

Static Keyword

1. Static Variable

When a variable is declared static in Java programming, it means that the variable belongs to the class itself rather than to any specific instance of the class. This means that there is only one copy of the variable in memory, regardless of how many instances of the class are created.

```
class student
{
    String name;
    int roll;
    static String faculty;

    public void show()
    {
        System.out.println(name + ":" + roll + ":" + faculty);
    }
}

public class static_variable
{
    Run | Debug
    public static void main (String args[])
    {
        student ob1 = new student();
        ob1.name = "Bipash";
        ob1.roll = 2;
        ob1.faculty = "BCA";

        student ob2 = new student();
        ob2.name = "Prabin";
        ob2.roll = 22;
        ob2.faculty = "BBA";

        ob1.faculty = "BCA 2020";

        ob1.show();
        ob2.show();
    }
}
```

2. Static Method

Any method that uses the static keyword is referred to as a static method.

Features of static method:

- A static method in Java is a method that is part of a class rather than an instance of that class.
- Every instance of a class has access to the method.
- Static methods have access to class variables (static variables) without using the class's object (instance).
- Only static data may be accessed by a static method. It is unable to access data that is not static (instance variables).
- In both static and non-static methods, static methods can be accessed directly.

```
class Student
{
    String name;
    int roll_no;
    static String faculty;

    public void show()
    {
        System.out.println( name + " : " + roll_no + " : " + faculty);
    }

    public static void show1(Student obj)
    {
        System.out.println( obj.name + " : " + obj.roll_no + " : " + faculty);
    }
}

class staticDemo
{
    Run | Debug
    public static void main(String args[])
    {
        Student obj1 = new Student();
        obj1.name = "MD";
        obj1.roll_no = 15;
        Student.faculty = "science";

        Student obj2 = new Student();
        obj2.name = "Bipash";
        obj2.roll_no = 10;
        Student.faculty = "management";

        Student obj3 = new Student();
        obj3.name = "Nitu";
        obj3.roll_no = 8;
        Student.faculty = "Humanities";

        Student.faculty = "BCA";

        Student.show1(obj1);
    }
}
```

Package in java

One way of reusing the code already created is by extending the classes and implementing the interfaces. But this is limited to reusing the classes within a program. What if we want to use classes from other programs without physically copying them into the program under development. This can be achieved in java by using the concept of packages.

Package is similar to header file and class libraries in other language. Package can be defined as a grouping of related types (classes, interfaces, enumerations and annotations) providing access protection and namespace management. Packages in java are classified in two categories.

1. User API Packages
2. User defined packages

User API Packages

Java API packages provided a large number of classes grouped into different packages according to functionality. Most of the time we use classes from Java API packages.

Figure below shows the java API packages that are frequently used in the programs:

Package	Description
<code>java.awt.event</code>	The Java Abstract Window Toolkit Event Package contains classes and interfaces that enable event handling for GUI components in both the <code>java.awt</code> and <code>javax.swing</code> packages.
<code>java.io</code>	The Java Input/Output Package contains classes and interfaces that enable programs to input and output data.
<code>java.lang</code>	The Java Language Package contains classes and interfaces (discussed bookwide) that are required by many Java programs. This package is imported by the compiler into all programs.
<code>java.util</code>	The Java Utilities Package contains utility classes and interfaces that enable such actions as date and time manipulations, random-number processing (class <code>Random</code>) and the storing and processing of large amounts of data.
<code>java.util.concurrent</code>	The Java Concurrency Package contains utility classes and interfaces for implementing programs that can perform multiple tasks in parallel.
<code>javax.swing</code>	The Java Swing GUI Components Package contains classes and interfaces for Java's Swing GUI components that provide support for portable GUIs.

The packages are organized in a hierarchical structure. Since the package is collection of either related classes and/or interface the packages contain class. For e.g. the package `awt` contains various classes required for implementing graphical user interface.

Accessing Packages:

1. If we want only use one class from package, we can do that by
import java.util.Scanner;
2. If we want to use many classes of a package then,
import java.util.*;

User Defined Packages

We can create our own package in java. To create package in java we must first declare the name of the package using the package keyword followed by package name and then we write the definition of the classes that we want to be in that package. For example, if we want class named FirstClass to be inside package firstPackage then we write:

```
package firstPackage;  
  
public class FirstClass  
  
{  
  
//body of class  
  
}
```

Example

Calculator.java

```
package tools;  
public class Calculator  
{  
    public int Add(int n1, int n2)  
    {  
        return n1 + n2;  
    }  
    public int Sun(int n1, int n2)  
    {  
        return n1 - n2;  
    }  
}
```

AdvanceCalculator.java

```
package tools;
public class AdvanceCalculator
{
    public int mul(int n1, int n2)
    {
        return n1 * n2;
    }
    public int div(int n1, int n2)
    {
        return n1 / n2;
    }
}
```

Demo.java

```
import tools.AdvanceCalculator;
import tools.Calculator;
public class Demo
{
    Run | Debug
    public static void main(String a[])
    {
        Calculator c1 = new Calculator();
        AdvanceCalculator c2 = new AdvanceCalculator();

        System.out.println(c1.Add(n1:2,n2:5));
        System.out.println(c2.div(n1:10,n2:5 ));
    }
}
```

Access Modifiers

Access control is a mechanism, an attribute of encapsulation which restricts the access of certain members of a class to specific parts of a program. Access to members of a class can be controlled using the access modifiers.

There are four types of access modifiers:

- **Private:** The access level of a private modifier is only within the class. It cannot be accessed from outside the class.
- **Default:** The access level of a default modifier is only within the package. It cannot be accessed from outside the package. If you do not specify any access level, it will be the default.
- **Protected:** The access level of a protected modifier is within the package and outside the package through child class. If you do not make the child class, it cannot be accessed from outside the package.
- **Public:** The access level of a public modifier is everywhere. It can be accessed from within the class, outside the class, within the package and outside the package.

Access Modifier Access Location	Public	Protected	Friendly	Private
Same class	Yes	Yes	Yes	Yes
Subclass in same package	Yes	Yes	Yes	No
Other classes in same package	Yes	Yes	Yes	No
Subclasses in other package	Yes	Yes	No	No
Non subclasses in other package	Yes	No	No	No

